Assignment 5

December 10, 2024

0.1 Problem 1

0.1.1 1a)

According to the model, a one-million dollar increase in healthcare expenditures would increase life expectancy by 0.3269 years.

0.1.2 1b)

All four total independent variable explain 69.9% of the data

0.1.3 1c)

When an observation has a value of 0 for all independent variables, the model predicts lifeex_total to be 78.3454 years.

0.1.4 1d)

A P-value of 0.000 allow us to safely reject the null hypothesis that there is no relationship between GDP and life expectancy. This means that the probability of seeing a coefficient of 0.1620 is 0% if the null hypothesis is true. The result is statistically important at 0.05

0.1.5 1e)

If the relationship between life expectancy in the population distribution that the data is drawn from follows a quadratic rather than a linear relationship, then this evidence would cast doubt on the validity of this model. Brief explanation: the validity of the p-value would be challenged. The reason is that in the residual v.s. fitted plot, it would display a pattern instead of being randomly spread out. With a small residual at the start and the end of both lines and a big residual in the middle of the line, heteroskedasticity may be present and that would affect the p-value.

I would address this problem by changing the type of regression. I would adjust the coefficient of whatever variables share a quadratic relationship with life expectancy and adjust the variable itself to be in accordance with the reported quadratic relationship(for instance GDP would become GDP^2 as suggested in the question).

0.1.6 1f)

Statistical approach

0.2 Problem 2

0.2.1 2a)

Observational study. The researcher could not set up a control group or treatment group for this experiment. They solely collect data and study the relationship between independent variables and dependent variable.

0.2.2 2b)

I think one of the most important assumptions about treatment and outcome we need to hold to inference an observational study casually is to assume that there are no confounder. We would also assume that endogeneity not be present in our treatment and outcome. We assume there is covariation between the treatment and outcome variables, and that the treatment precedes the outcome.

0.2.3 2c)

One potential reciprocal causal relationship could be between GDP and spending on healthcare. The increase in GDP means the government would have more income through taxes comparatively speaking, which means that it is probable that the government would increase the budget for healthcare since it is one of many important sectors that people extremely care about. Better coverage in healthcare could lead to a smaller mortality rate or shorter sick leaves, meaning more individuals would be more likely to participate in economic activities leading to a higher GDP.

0.2.4 2d)

Confounders are a concern in this study. Population structure could be a confounder for this study. Countries with a high proportion of older people may limit the growth of GDP since a lot of them are retired and no longer able to work. On the other hand, a population structure with a high proportion of elderly also lowers the life expectancy of that country because the mortality rate is a lot higher for the elderly.

0.2.5 2e)

The collider is the Bellevue Hospital. It is a Level 1 trauma center, meaning it is capable of handling the most serious injury or health problems. People from different social economic classes are all present in the hospital because of its resources of experienced doctors and cutting-edge research. Rich patients would choose Bellevue no matter the severity of their illness because they could afford the best care. The middle class would also come to Bellevue if their condition is incurable for many smaller hospitals. The lower class would also come to Bellevue for their newer treatment unavailable at smaller hospitals by taking out loans or other economic means to help them afford their treatment.

0.3 Problem 3

0.3.1 3a)

The difference between supervised and unsupervised training lies in the labeling of the data. Supervised data are labeled, while unsupervised data are not labeled. Meaning that we already have some sort of categorization for both input and output in a supervised training. On the other hand,

when we want the machine to discover some pattern and categorize data into a cluster to give us some insight, we use unsupervised training.

0.3.2 3b)

While both RMSE and R-square measure how well the model fits the data in some sense, their unit of measurement is quite different, which reflects different aspects of the model. RMSE is calculated by square root the sum of the difference between prediction and observation divided by the total number of data. RMSE shows the average error of the model and the result is in the unit of dependent variable. R-square on the other hand, is calculated through 1-TSS/RSS. R-square gives a number to show that how much of the variance is explained by the model in percentage.

0.3.3 3c)

We usually don't desire perfect accuracy because perfect accuracy usually comes with overfitting, which strips away the model's ability to handle new data. When a model overfit, it captures the random noise that is specific to the training data set, which would ultimately lead to poor generalization of new data.

0.4 Problem 4

[1]:
```
## Import Statement
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sklearn
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import mean_squared_error
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn import metrics
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

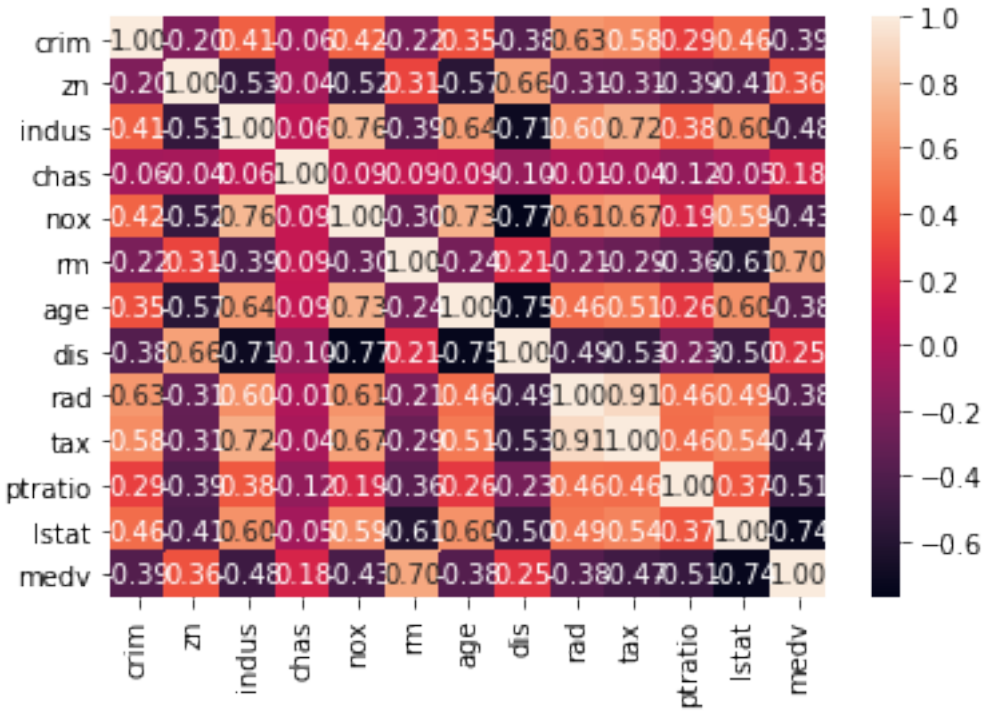
[2]:
```
## Basic Info about dataset
df = pd.read_csv('shared/data/neighborhoods_boston.csv')
auxiliary_df = pd.read_csv('shared/data/vars_boston.csv')
print(df.info)
pd.set_option("display.max_colwidth", 1000)
print(auxiliary_df.info)
```

```
<bound method DataFrame.info of         crim    zn  indus  chas    nox     rm
age     dis  rad  tax  \
0    0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296
1    0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242
2    0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242
3    0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222
```

```
4    0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222
..       …   …    …   …    …    …   …      …  …  …
501  0.06263   0.0  11.93     0  0.573  6.593  69.1  2.4786    1  273
502  0.04527   0.0  11.93     0  0.573  6.120  76.7  2.2875    1  273
503  0.06076   0.0  11.93     0  0.573  6.976  91.0  2.1675    1  273
504  0.10959   0.0  11.93     0  0.573  6.794  89.3  2.3889    1  273
505  0.04741   0.0  11.93     0  0.573  6.030  80.8  2.5050    1  273

     ptratio  lstat  medv
0       15.3   4.98  24.0
1       17.8   9.14  21.6
2       17.8   4.03  34.7
3       18.7   2.94  33.4
4       18.7   5.33  36.2
..       …    …   …
501     21.0   9.67  22.4
502     21.0   9.08  20.6
503     21.0   5.64  23.9
504     21.0   6.48  22.0
505     21.0   7.88  11.9

[506 rows x 13 columns]>
<bound method DataFrame.info of    Unnamed: 0  \
0        crim
1          zn
2       indus
3        chas
4         nox
5          rm
6         age
7         dis
8         rad
9         tax
10    ptratio
11      lstat
12       medv

                                                 Description
0                                  Per capita crime rate by town.
1         Proportion of residential land zoned for lots over 25,000 sq.ft
2                      Proportion of non-retail business acres per town.
3   Charles river dummy variable (= 1 if tract bounds river; 0 otherwise).
4                   Nitrogen oxides concentration (parts per 10 million).
5                             Average number of rooms per dwelling.
6                Proportion of owner-occupied units built prior to 1940.
7           Weighted mean of distances to five boston employment centres.
8                           Index of accessibility to radial highways.
9                            Full-value property-tax rate per $10,000.
```

```
10                                          Pupil-teacher ratio by town.
11                                Lower status of the population (percent).
12                              Median value of owner-occupied homes in $1000s.  >
```

[3]:
```
### 4a)
corr = df.corr()
sns.heatmap(corr,annot=True, fmt='0.2f')
plt.show()
```

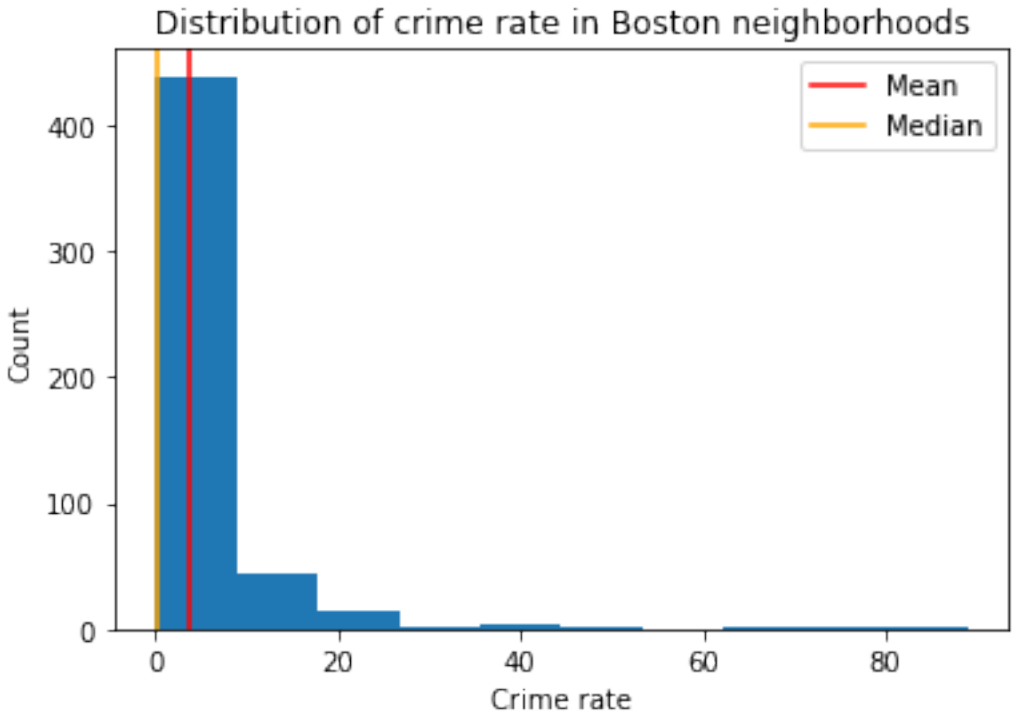


0.4.1 4b)

It is a little counterintuitive that tax and crime rates are positively correlated. An increase in property tax is associated with an increase in crime rate. We would expect that a higher property tax region would make more money, which would lead to better funds for the police force and a lower crime rate.

[4]:
```
### 4c)
plt.hist(df['crim'])
plt.axvline(x=df['crim'].mean(),color='red',label='Mean')
plt.axvline(x=df['crim'].median(),color = 'orange',label='Median')
plt.title("Distribution of crime rate in Boston neighborhoods")
plt.xlabel("Crime rate")
plt.ylabel("Count")
plt.legend()
```

```
plt.show()
```



0.4.2 4c)

In the graph, the mean is greater than the median, meaning that this is a right-skewed graph. In the context, this means that the majority of the neighborhoods have relatively low crime rate(somewhere between 0-10%), while there are some outliers with extremely high crime rate dragging the mean to the right.

[5]:
```
### 4d)

crim_over_thresh = (df['crim'] >= df['crim'].median()).astype(int)
```

0.4.3 4d)

From what we have concluded from 4c), this is a right-skewed graph. From what we learned about median and mean, we know that the median is a lot more robust, meaning that it is not easily affected by outliers. Setting the median as the threshold would allow more data to be pulled into the sample, which would also be more representative of the true population.

0.4.4 4e)

[6]:
```
X_c = df[['zn','indus','chas','nox', 'rm',␣
 ↪'age','dis','rad','tax','ptratio','lstat','medv']]
x_train,x_val,y_train,y_val = train_test_split(X_c,crim_over_thresh,test_size=0.
 ↪2,random_state=121224)
x_val,x_test,y_val,y_test= train_test_split(x_val,y_val,test_size=0.
 ↪5,random_state=121224)
```

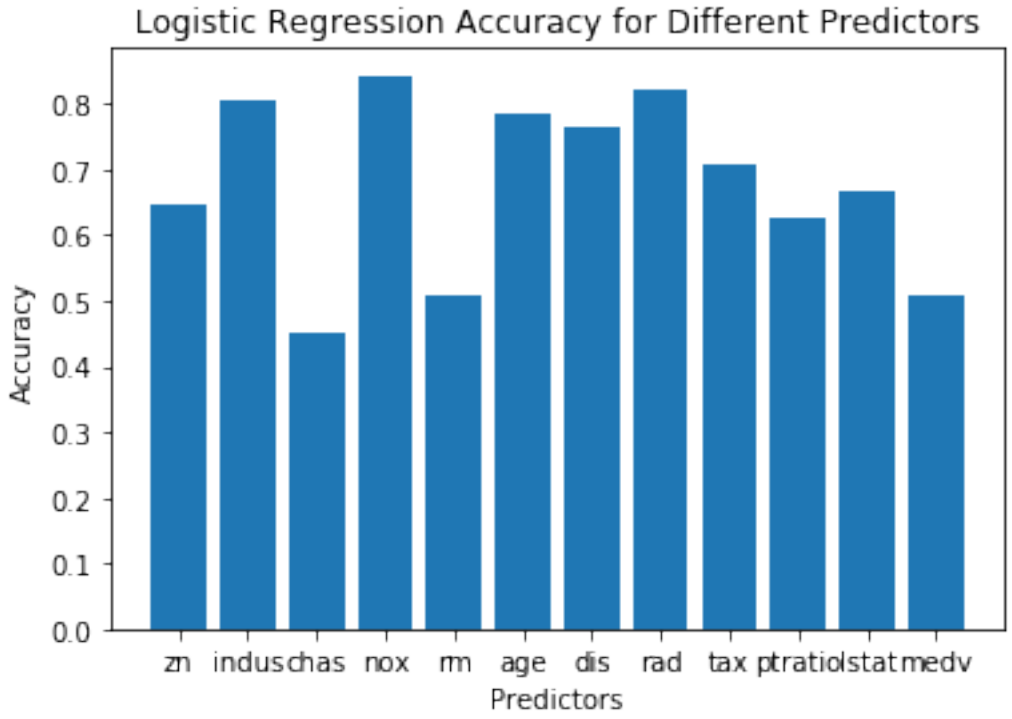
0.4.5 4f)

[7]:
```
result = []
model = LogisticRegression()
for predictors in X_c.columns:
    Train = x_train[[predictors]]
    Val = x_val[[predictors]]

    model.fit(Train,y_train)
    y_pred = model.predict(Val)
    accuracy = accuracy_score(y_val, y_pred)
    result.append([predictors,accuracy])


result = pd.DataFrame(result)
print(result.head(12))
plt.bar(result.iloc[:,0],result.iloc[:,1])
plt.xlabel('Predictors')
plt.ylabel('Accuracy')
plt.title('Logistic Regression Accuracy for Different Predictors')

## Best Model
model = LogisticRegression()
Train = x_train[['nox']]
model.fit(Train,y_train)
```

```
          0         1
0        zn  0.647059
1     indus  0.803922
2      chas  0.450980
3       nox  0.843137
4        rm  0.509804
5       age  0.784314
6       dis  0.764706
7       rad  0.823529
8       tax  0.705882
9   ptratio  0.627451
10    lstat  0.666667
```

```
11      medv  0.509804
```

[7]:
```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
                   intercept_scaling=1, l1_ratio=None, max_iter=100,
                   multi_class='auto', n_jobs=None, penalty='l2',
                   random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
                   warm_start=False)
```

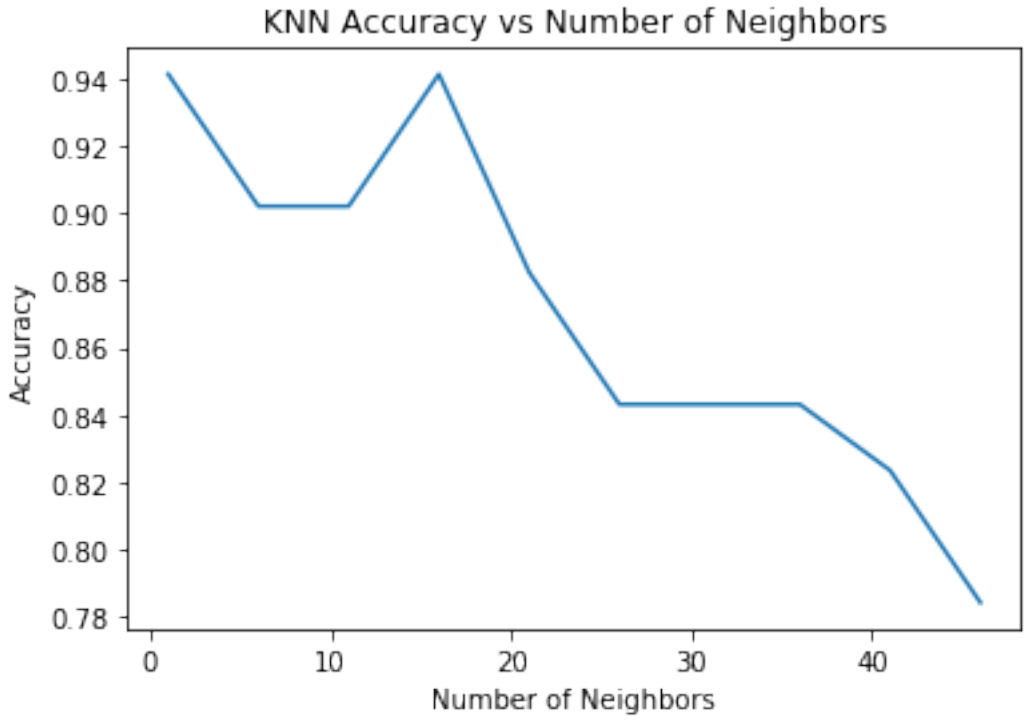


0.4.6 4g)

[8]:
```
accuracies = []
Neighbors = []
for i in range (0,50):
    if i%5 == 0:
        Neighbors.append(i+1)

for n in Neighbors:
    knn = KNeighborsClassifier(n_neighbors=n)
    knn.fit(x_train[['nox']], y_train)
    predictions = knn.predict(x_val[['nox']])
    accuracies.append(accuracy_score(y_val, predictions))
print(Neighbors)
print(accuracies)
```

```
# Plot the results
plt.plot(Neighbors, accuracies)
plt.xlabel("Number of Neighbors")
plt.ylabel("Accuracy")
plt.title("KNN Accuracy vs Number of Neighbors")
plt.show()

## Best Model
knn = KNeighborsClassifier(n_neighbors=16)
knn.fit(x_train[['nox']], y_train)
```

```
[1, 6, 11, 16, 21, 26, 31, 36, 41, 46]
[0.9411764705882353, 0.9019607843137255, 0.9019607843137255, 0.9411764705882353,
0.8823529411764706, 0.8431372549019608, 0.8431372549019608, 0.8431372549019608,
0.8235294117647058, 0.7843137254901961]
```



[8]:
```
KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                     metric_params=None, n_jobs=None, n_neighbors=16, p=2,
                     weights='uniform')
```

0.4.7 4h)

The clear winner is definitely KNeighbors model. The model has a much higher recall rate for detecting neighborhoods over the threshold compare to the logistic regression model, which means

it does not mistakenly predict a crime under the threshold neighborhood as a crime over the threshold neighborhood. The precision score for KNeighbors model is also a lot higher for the prediction of both under or over the threshold.

[10]:
```
knn = KNeighborsClassifier(n_neighbors=16)
knn.fit(x_train[['nox']], y_train)
KNNPredictions = knn.predict(x_test[['nox']])

model = LogisticRegression()
Train = x_train[['nox']]
model.fit(Train,y_train)

LogisticPrediction = model.predict(x_test['nox'].values.reshape(-1,1))
print('KNN Classification Report')
print(classification_report(y_test,KNNPredictions))
print('Logistic Regression Classification Report')
print(classification_report(y_test,LogisticPrediction))
```

```
KNN Classification Report
              precision    recall  f1-score   support

           0       1.00      0.88      0.93        24
           1       0.90      1.00      0.95        27

    accuracy                           0.94        51
   macro avg       0.95      0.94      0.94        51
weighted avg       0.95      0.94      0.94        51

Logistic Regression Classification Report
              precision    recall  f1-score   support

           0       0.72      0.88      0.79        24
           1       0.86      0.70      0.78        27

    accuracy                           0.78        51
   macro avg       0.79      0.79      0.78        51
weighted avg       0.80      0.78      0.78        51
```

0.4.8 4i)

The k-nearest model is a single-variable predictor, which means that the model only uses one independent variable, nox, to predict the outcome. I think this algorithm works pretty well because nox is highly correlated with the per capita crime rate, which could mean that the data set has a nice boundary for classification. Another reason why the k-nearest model is doing relatively well is because the data is also heavily skewed with the majority in the lower crime rate.